

From empty repo to development-ready

The initiation workflow, one step at a time. Each step is one Claude command, run in its own fresh session **from the kit**. It produces one document in your **target repo**, and that document is final only when you merge its PR/MR to `main`.

- Kit · Claude session (control plane)
- Target repo clone
- You (approve, review, merge)
- Git host · PR/MR

Two repos, one operator

You run every command from the kit. The kit's guides, runner and templates never get copied into your project; only the documents you produce land there.

KIT · run commands here Control plane Commands, step guides, templates, step registry, role files. <code>.claude/commands/proj-init-*</code> <code>docs/guides/proj-init/</code> <code>.proj-init/state.json</code>	TARGET · registered by Step-00 Your project clone Every git operation and every produced document lands here. <code>CONTRIBUTING.md</code> · <code>README.md</code> <code>docs/PRODUCT.md</code> ... <code>docs/BACKLOG.md</code> AI tool adapters	GIT HOST · GitHub, Azure DevOps, GitLab, Bitbucket Review and merge One branch and one PR/MR per step. Merge to <code>main</code> = final. <code>init/<step></code> → PR/MR → <code>main</code> <code>gh</code> · <code>az repos</code> · <code>glab</code> · <code>web UI</code>
--	---	--

Where a document sits

No draft files and no status flags. A doc on a branch is a draft; a doc on `main` is final.

Not started Not on <code>main</code>, no branch —	Draft On <code>init/<step></code> in the target <code>/proj-init-<step></code>	In review · you Pushed after your yes, PR/MR open <code>gh pr create</code> · <code>az repos pr create</code> · <code>glab mr create</code>	Final · you Merged after the self-review checklist <code>merge to main</code>
---	--	---	---

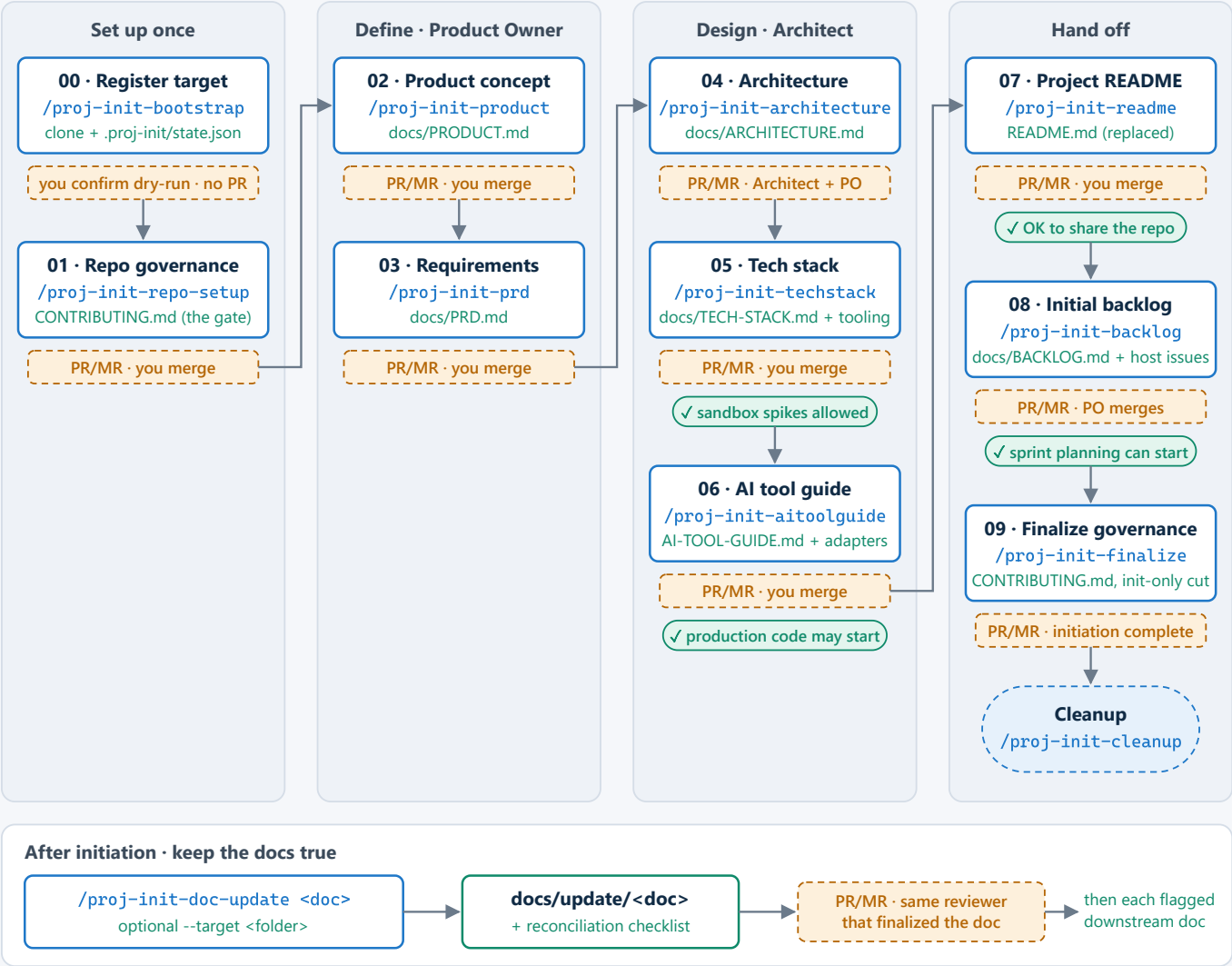
Start every step the same way

TERMINAL	<pre># from the kit folder – never from the target clone cd <kit-folder> claude</pre>
CLAUDE	<pre>/proj-init-<step> # e.g. /proj-init-product</pre>

One step, one session. For the next step, run `/clear` or start a new `claude` session. Steps 01–09 fast-forward `main` and run the status check themselves; run `/proj-init-doc-status` any time to see what's merged, what's open, and what's next. GitHub Copilot Chat users run the same `/proj-init-*` names from `.github/prompts/`.

The whole flow

Two one-time setup steps, then one document per step. Every step from 01 to 09 ends in a PR/MR that you merge. A step won't start until the documents it depends on are on `main`.



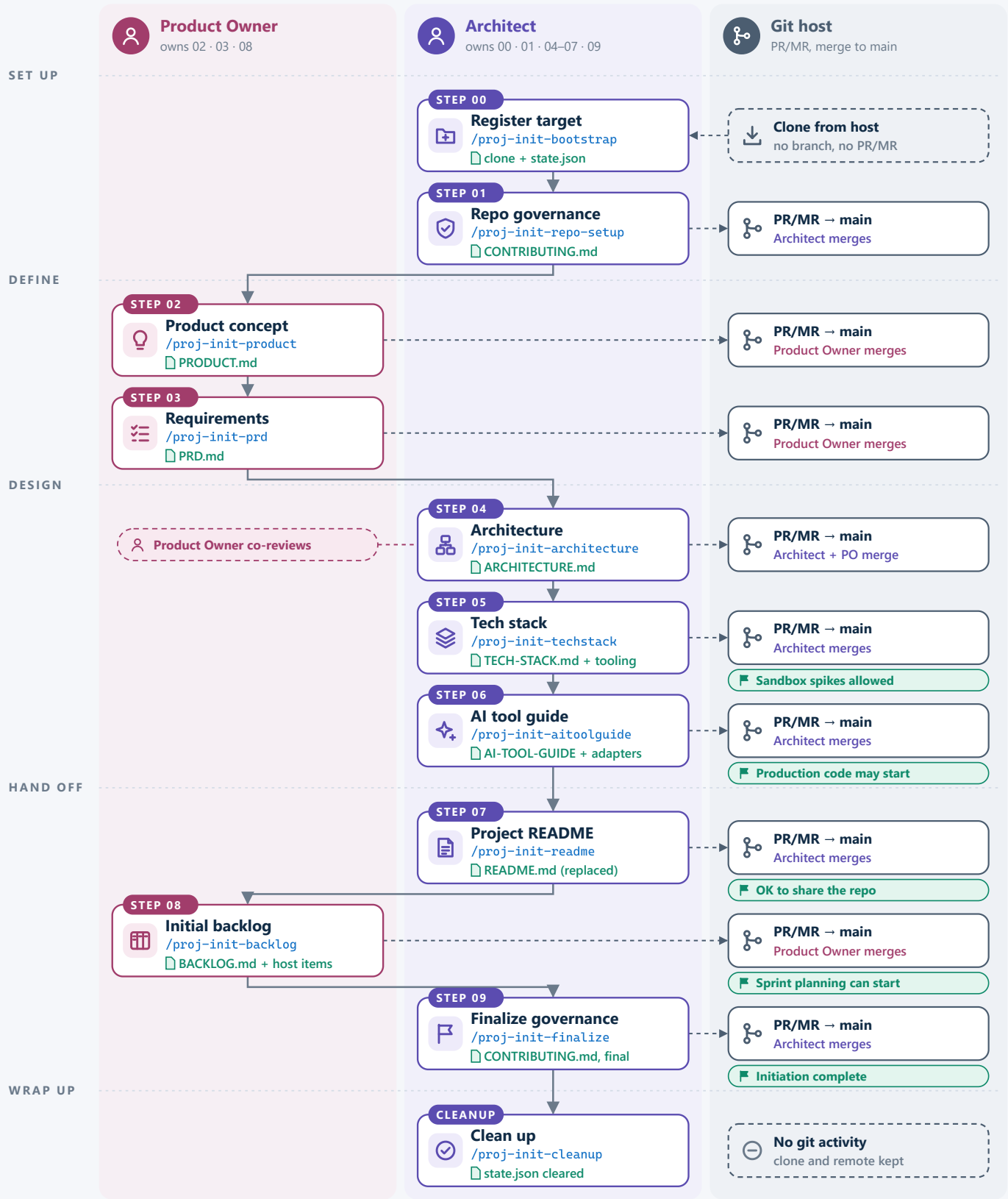
What each step waits for

The runner checks these on the target's `main` before it creates a branch. If any is missing, it stops and names it.

Step	Needs on main	Step	Needs on main
01	Step-00 registration only	06	ARCHITECTURE.md , TECH-STACK.md
02	CONTRIBUTING.md (Step-01 gate)	07	PRODUCT , PRD , ARCHITECTURE , TECH-STACK , AI-TOOL-GUIDE
03	PRODUCT.md	08	all of 07's inputs + README.md
04	PRD.md	09	BACKLOG.md
05	PRD.md , ARCHITECTURE.md	Cleanup	BACKLOG.md , init-only block gone from CONTRIBUTING.md

The whole flow, by who does what

The same flow, read top to bottom. Each step sits in the lane of the role that owns it; an elbow in the arrow is a handoff. The right column shows each PR/MR, who merges it, and what that merge unlocks.



- Command · Claude session in the kit
- Written to the target

Docs land under docs/ in the target; README.md and CONTRIBUTING.md stay at the root. Solo team? Wear the hat of the lane the step sits in.

Step by step

Run every command from the kit folder, one per fresh session. Steps 01–09 share one runner: resolve the target, fast-forward `main`, check status and preconditions, wait for your **yes**, branch, interview you one question at a time, draft, commit, then push and open the PR/MR only after a second **yes**.

00

Register the target repo

• Kit

• You confirm

Clones your project repo and records where it lives, so every later step operates on that clone. Nothing is committed or pushed.

CLAUDE

/proj-init-bootstrap

ASKS

The target's git URL, and an empty or non-existent local folder to clone into.

RUNS

`node scripts/bootstrap-target-repo.mjs --target <folder> --url <git-url>` as a dry run, then again with `--apply`

WRITES

The clone at `<folder>`, plus `.proj-init/state.json` in the kit (gitignored, local to this machine)

GATE

■ You review the dry run before `--apply`. Only one workspace at a time; replace it only with `--force`.

GIT

No branch, no commit, no PR/MR.

01

Set up repo governance

• Kit

• Target

• You merge

The gate every later step relies on: the branching convention, the commit convention, and the self-review checklist.

CLAUDE

/proj-init-repo-setup

OWNER

Architect · reviewed by Architect

WRITES

`CONTRIBUTING.md`: permanent rules plus an `INITIATION-ONLY` fenced block, and branch protection on `main` if your host and plan allow it

GIT

`init/repo-setup` · docs: add repo governance baseline (Step-01)

GATE

■ Self-review checklist, then you merge. Every later step stops until `CONTRIBUTING.md` is on `main`.

02

Product concept

• Kit

• Target

• You merge

What we are building, and why.

CLAUDE

/proj-init-product

OWNER

Product Owner · reviewed by Product Owner

READS

`CONTRIBUTING.md` on `main` (the Step-01 gate)

WRITES

`docs/PRODUCT.md`

GIT

`init/product` · docs: add `PRODUCT.md` (Step-02)

GATE

■ The Product Owner approves and merges.

03

Product requirements

● Kit

● Target

● You merge

Turns the concept into testable requirements.

CLAUDE

/proj-init-prd

OWNER

Product Owner · reviewed by Product Owner

READS

docs/PRODUCT.md

WRITES

docs/PRD.md

GIT

init/prd · docs: add PRD.md (Step-03)

GATE

■ The Product Owner approves and merges.

04

Architecture

● Kit

● Target

● You merge

System structure and design decisions. The runner loads both the Architect and Product Owner role context.

CLAUDE

/proj-init-architecture

OWNER

Architect · reviewed by Architect + Product Owner

READS

docs/PRD.md

WRITES

docs/ARCHITECTURE.md

GIT

init/architecture · docs: add ARCHITECTURE.md (Step-04)

GATE

■ The Architect and Product Owner both approve, then merge.

05

Tech stack

● Kit

● Target

● You merge

Locks the approved technologies and adds the build/test/lint commands to CONTRIBUTING.md .

CLAUDE

/proj-init-techstack

OWNER

Architect · reviewed by Architect

READS

docs/PRD.md , docs/ARCHITECTURE.md

WRITES

docs/TECH-STACK.md ; appends the tooling layer to CONTRIBUTING.md and leaves the Step-01 governance layer intact

GIT

init/techstack · docs: add TECH-STACK.md and CONTRIBUTING tooling layer (Step-05)

GATE

■ The Architect approves and merges.

AFTER MERGE

Sandbox spikes are allowed, on isolated spike branches that never merge to main .

06

AI tool guide

● Kit

● Target

● You merge

The rules every AI tool follows on this project. Shared rules live in one guide; each adapter carries only what's specific to its tool.

CLAUDE

/proj-init-aitoolguide

OWNER

Architect · reviewed by Architect

ASKS

Which AI tools the team will use. This is asked before any adapter is generated.

READS

docs/ARCHITECTURE.md , docs/TECH-STACK.md

WRITES

docs/AI-TOOL-GUIDE.md every time, plus adapters only for the tools you picked, e.g. CLAUDE.md or .github/copilot-instructions.md

GIT

init/aitoolguide · docs: add AI-TOOL-GUIDE.md and tool adapters (Step-06)

GATE

■ The Architect approves and merges.

AFTER MERGE

Production implementation can start.

07

Project README

● Kit

● Target

● You merge

Replaces the target's boilerplate README with a real entry point: setup, environment config, and how to run.

CLAUDE

/proj-init-readme

OWNER

Architect · reviewed by Architect

READS

PRODUCT.md , PRD.md , ARCHITECTURE.md , TECH-STACK.md , AI-TOOL-GUIDE.md

WRITES

README.md , replacing the existing file with all placeholders removed

GIT

init/readme · docs: replace boilerplate README with project README (Step-07)

GATE

■ Every command in the README is tested before commit. The Architect approves and merges.

AFTER MERGE

Required before the repo is shared more widely.

08

Initial backlog

● Kit

● Target

● Host

● You merge

Epics and stories derived from the product and requirements docs, seeded into your issue tracker.

CLAUDE

/proj-init-backlog

OWNER

Product Owner · reviewed by Product Owner

READS

The five docs Step-07 reads, plus README.md

WRITES

docs/BACKLOG.md , plus host work items via gh or az boards ; with neither CLI available, PENDING IDs and manual creation notes

GIT

init/backlog · docs: add BACKLOG.md – Step-08 initial backlog

GATE

■ You approve the derived backlog before anything is written to the host or to disk. Items are then created one at a time. The Product Owner merges.

AFTER MERGE

Required before sprint planning. Next: Step-09.

09

Finalize governance

● Kit

● Target

● You merge

Removes the initiation-only rules from `CONTRIBUTING.md` and hands the team a permanent contribution core.

CLAUDE

/proj-init-finalize

OWNER

Architect · reviewed by Architect

READS

`docs/BACKLOG.md` on `main` (the gate) and `CONTRIBUTING.md` on `main`

WRITES

`CONTRIBUTING.md`: the `INITIATION-ONLY` block is removed (markers included) and a `DEVELOPMENT-PHASE-GOVERNANCE` stub is inserted. Permanent rules and the tooling layer stay verbatim.

GIT

`init/finalize` · `docs: finalize CONTRIBUTING.md for the development phase (Step-09)`

GATE

■ Stops if the `INITIATION-ONLY` markers are missing. Once the Architect merges, initiation is complete.

✓

Clean up the workspace

● Kit

● You confirm

Unregisters the workspace from the kit. The target clone and its remote are left untouched.

CLAUDE

/proj-init-cleanup

CHECKS

`docs/BACKLOG.md` is on `main` and `INITIATION-ONLY` is gone from `CONTRIBUTING.md`

RUNS

`node scripts/bootstrap-target-repo.mjs --clear`, after your **yes**

WRITES

Deletes `.proj-init/state.json`. Nothing else.

GIT

None.

Utilities, any time

CLAUDE

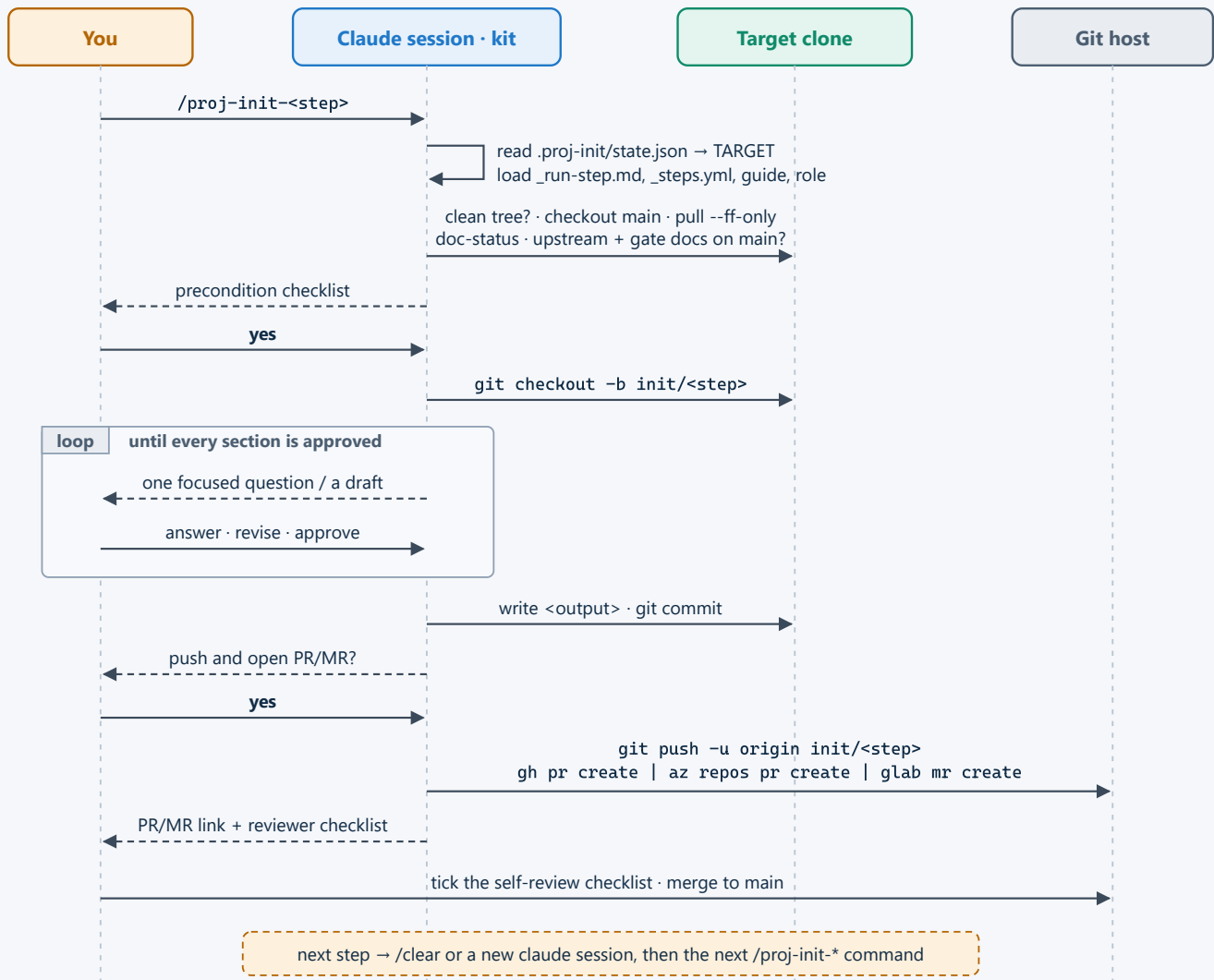
```
/proj-init-doc-status           # read-only: merged, open PRs/MRs, what's next
/proj-init-doc-update <doc>    # e.g. PRD.md → PR/MR + reconciliation checklist
# add --target <folder> to either for a finished or other project
```

Which document to update

If this happens	Run <code>/proj-init-doc-update</code> on
Product scope, goal, or target users change	<code>PRODUCT.md</code>
Requirements or acceptance criteria change	<code>PRD.md</code>
System design or a major architecture decision changes	<code>ARCHITECTURE.md</code>
A library, framework, or tool is added, removed, or version-locked	<code>TECH-STACK.md</code>
AI tool rules, banned patterns, or scope boundaries change	<code>AI-TOOL-GUIDE.md</code>
Setup steps, env config, or run commands change	<code>README.md</code>
Epics or stories added, removed, rescoped, or host IDs change	<code>BACKLOG.md</code>

One step up close

What happens between you, the Claude session in the kit, the target clone, and your Git host during any step from 01 to 09.



After the PR/MR opens

From here on it's your job. The command's job ended when the PR/MR opened.

1. **Give it fresh eyes.** Wait at least a few hours, ideally a full day, before reviewing.
2. **Tick the self-review checklist** in the PR/MR description: every required section is covered, no upstream doc changed after you branched, and the change went through a PR/MR, not a direct push.
3. **Merge to `main`.** Merging is what makes it final. Step-04 also needs the Product Owner's sign-off.
4. **Start the next step in a fresh session.** It fast-forwards `main` in the target before it checks that the upstream documents are merged.

Rules that hold at every step

One step, one fresh session. Never run two step commands in the same session. Status is checked first, then only the step you asked for runs.	Run from the kit, write to the target. Commands load guides from the kit and do git work in the registered clone. Kit files are never copied in.
Merge = final. A doc on a branch is a draft; a doc on <code>main</code> is final. There are no draft files and no status flags.	Nothing ships without your yes. Explicit confirmation before branching, before every host write, and before every push.
Never on <code>main</code>. No commits or pushes to <code>main</code> , and no force-pushes to shared branches. Every change goes through a PR/MR.	Upstream first. A step stops if any doc it depends on isn't on <code>main</code> , or if its own output is already final (use <code>/proj-init-doc-update</code> instead).
Same machine, same operator. <code>.proj-init/state.json</code> is local to one machine. To continue elsewhere, re-run Step-00.	Never assume an answer. Claude asks one focused question at a time until no template placeholder is left.

Milestones

AFTER 05 MERGES Spikes Isolated spike branches only, never merged to <code>main</code> .	AFTER 06 MERGES Production code Implementation work can begin.	AFTER 07 MERGES Share the repo The README is the real entry point.	AFTER 08 MERGES Sprint planning Then Step-09 completes initiation.
--	--	--	--

Command cheat sheet

Step	Claude command	Produces in the target	Merged by
00	<code>/proj-init-bootstrap</code>	clone + kit-local <code>.proj-init/state.json</code>	— (you confirm)
01	<code>/proj-init-repo-setup</code>	<code>CONTRIBUTING.md</code> (governance)	Architect
02	<code>/proj-init-product</code>	<code>docs/PRODUCT.md</code>	Product Owner
03	<code>/proj-init-prd</code>	<code>docs/PRD.md</code>	Product Owner
04	<code>/proj-init-architecture</code>	<code>docs/ARCHITECTURE.md</code>	Architect + PO
05	<code>/proj-init-techstack</code>	<code>docs/TECH-STACK.md</code> + tooling layer	Architect
06	<code>/proj-init-aitoolguide</code>	<code>docs/AI-TOOL-GUIDE.md</code> + adapters	Architect
07	<code>/proj-init-readme</code>	<code>README.md</code>	Architect
08	<code>/proj-init-backlog</code>	<code>docs/BACKLOG.md</code> + host work items	Product Owner
09	<code>/proj-init-finalize</code>	<code>CONTRIBUTING.md</code> (init-only removed)	Architect
—	<code>/proj-init-cleanup</code>	nothing; unregisters the workspace	— (you confirm)

Solo / process-enforced: one person may wear both hats; take on the one each step calls for. Source of truth: docs/guides/proj-init/(_steps.yml, _run-step.md, step guides).